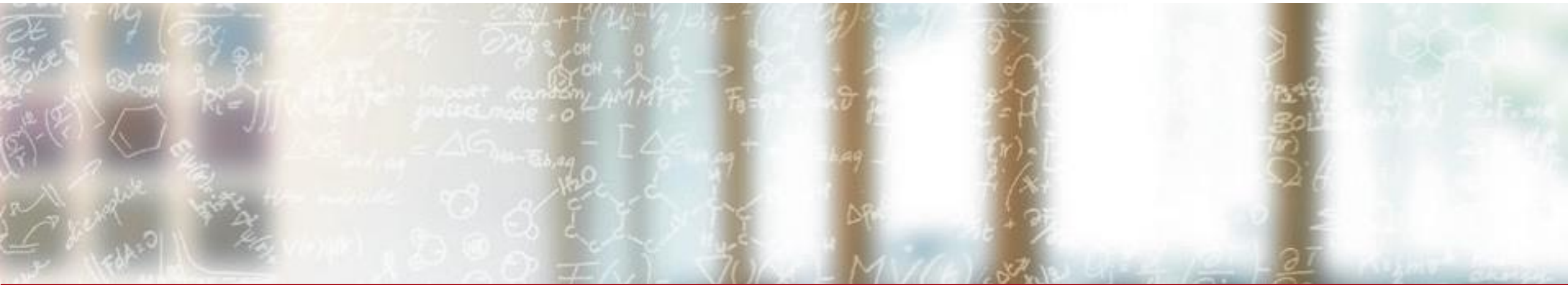




CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



Storage at CSCS

CSCS User Lab Day 2026

Dr. Matthias Kraushaar, CSCS

August 28, 2026

What this talk covers

1. Storage Awareness & Workflow Integration

- Storage is a system, not just space
- Storage tiers and how they map to your workflow

2. Understanding the File Systems

- How the file systems work (at a high level)
- Sources for potential performance issues

3. Storage Best Practices

- Striping and other tuning options
- Handling many small files
- Quotas and how to avoid surprises
- Data transfers: Globus, planning, automation.

Storage Awareness & Workflow Integration

Storage: More than just disk space

- Storage is not a passive resource, it's a system with architecture, behaviour, and performance characteristics that directly affect your scientific workflow.
 - Not all storage is equal, different tiers serve different purposes.
 - Efficiency requires understanding how files are stored, accessed, and moved
 - Planning matters: storage should be budgeted and tested like compute

To use storage efficiently, you need to get a basic understanding how it works, not just how much your project got allocated.

Storage Tiers

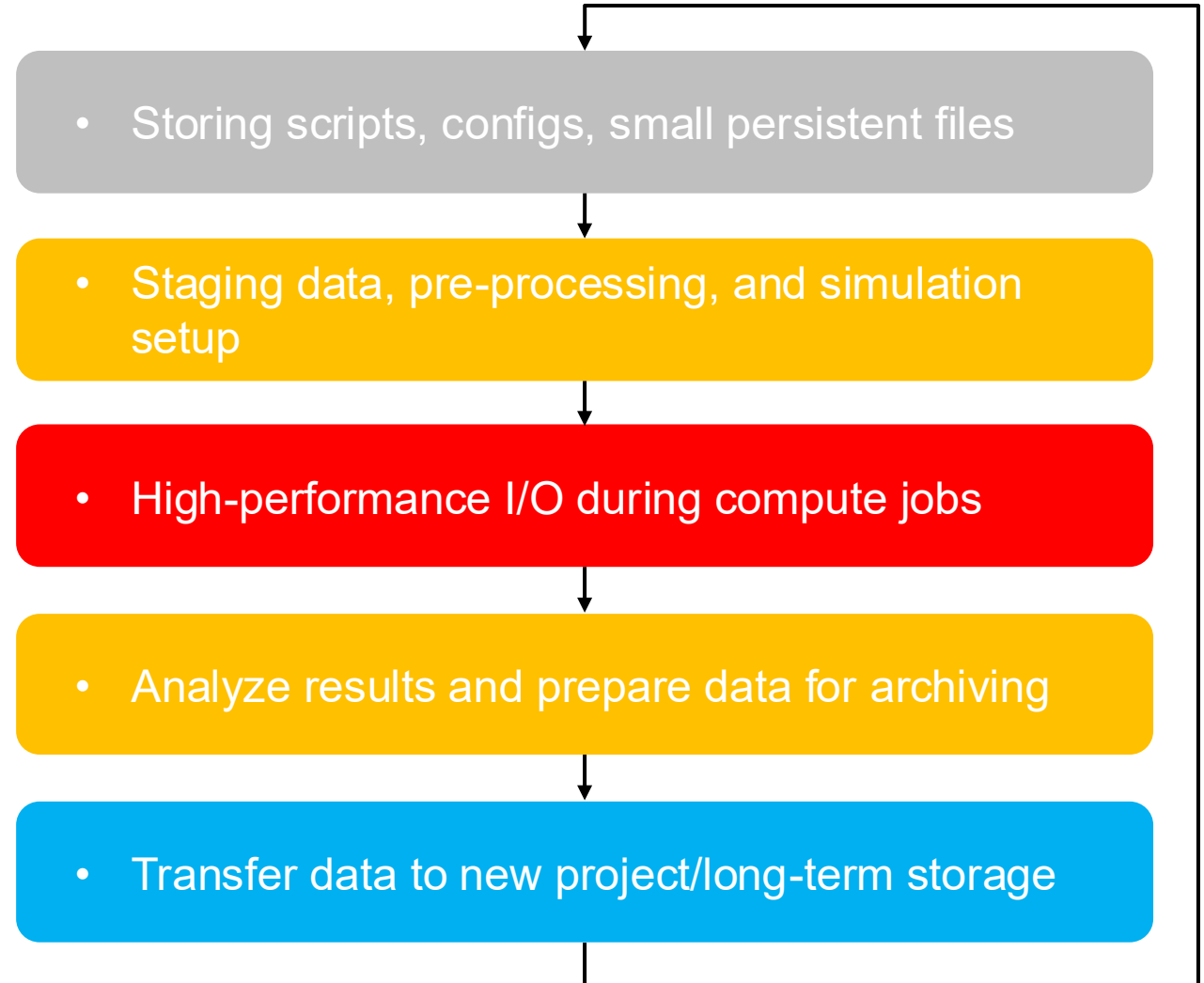
Tier	File system	Storage Systems
Hot	\$SCRATCH	Capstor, lopsstor, Ritom
- High-performance, short-term active data ➤ Here you can actively tune I/O performance through system-level features!		
Warm	\$STORE, \$SCRATCH	Capstor, Ritom
- Medium-term shared data and staging ➤ Workflow optimization is key here!		
Cold	External archive	Tape, Ceph (S3)
- Long-term data preservation ➤ Plan early for data reduction and transfer logistics!		

Outside of the tiers

User Home	\$HOME	Vadret
Small personal files, source code, configuration files, ...		

Mapping storage tiers to a typical HPC workflow

1. Project Initialization
(\$HOME is not part of the storage tiers!)
2. Data Preparation
\$STORE/\$SCRATCH
3. Simulation execution
\$SCRATCH
4. Post-processing & Analysis
\$STORE/\$SCRATCH
5. Archiving & Sharing



The Storage Systems behind the Names

Storage System	Type	File System	Technology	Optimized for	Cleanup
Capstor	HPE ClusterStor	Lustre	HDD, O(100PB)	Outputs involving large, contiguous I/O	30 days
lopsstor	HPE ClusterStor	Lustre	NVMe SSD, O(3PB)	Many small, random I/O operations	14 days
Ritom	VAST	VAST over NFS	SSD, O(10PB)	Many ranks writing one shared file	30 days
Vadret	VAST	VAST over NFS	SSD, O(1PB)	Serving \$HOME, Not intended for computation!	-

- Capstor, lopsstor, and Ritom are accessible from the high-speed Slingshot network on Alps.
 - **Avoid** using **\$HOME** in SLURM jobs!

Quotas

- Quotas limit Capacity (GB/TB) and inodes (number of files and directories).
- Typical user quotas:

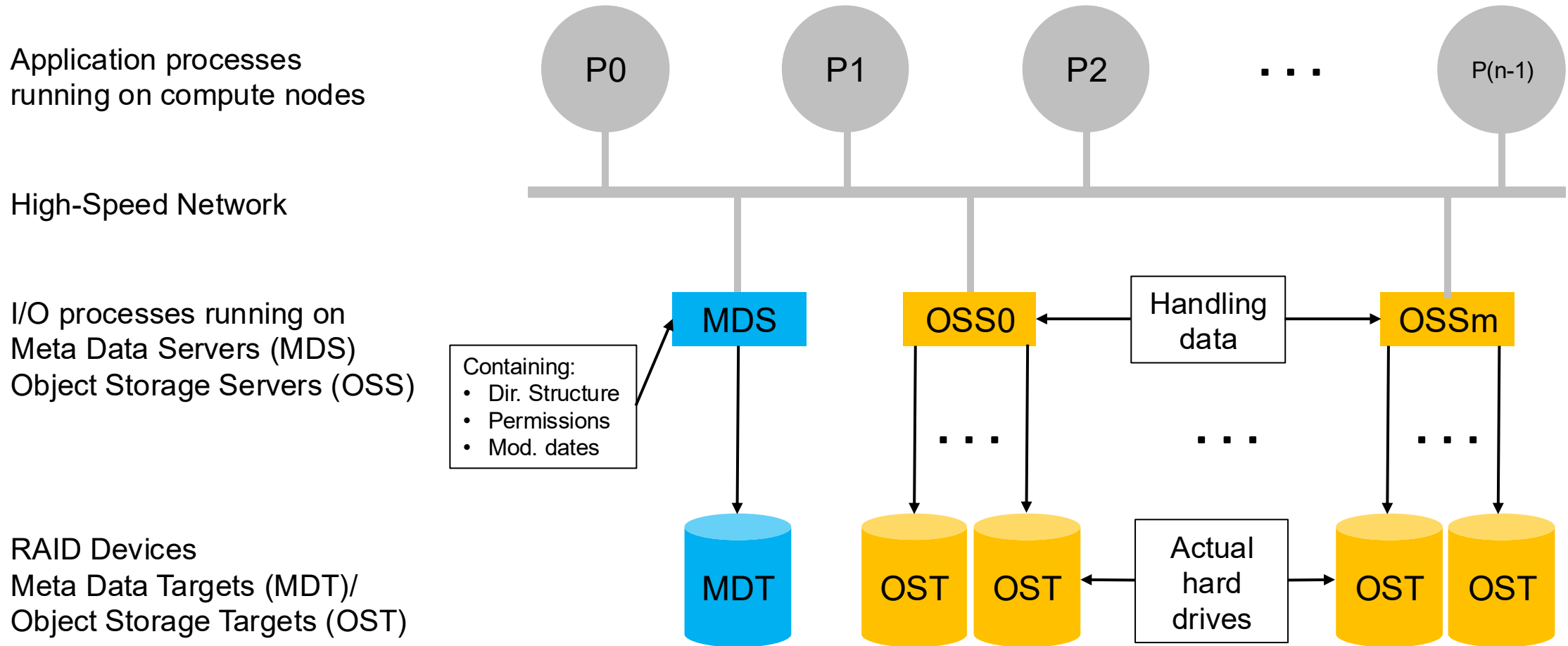
File System	Capacity quota	inode quota
\$HOME	50GB	500 000
\$SCRATCH	150TB	1 000 000
\$STORE	(allocated per project)	

- Check usage of user/project with the **quota** command on:
 - ela.cscs.ch (CSCS front-end)
 - Login nodes of Daint, Eiger, Santis, Clariden
- Exceeding the quota
 - \$HOME: blocks further I/O on that file system and can create login issues
 - \$SCRATCH:
 - Grace period of 1 week, after that job submission is no longer possible!

Understanding the File Systems

The Lustre-based file systems at CSCS

Simplified Lustre Overview



Lustre Architecture: Where Bottlenecks Can Occur

- **Metadata Server (MDS):**
 - Bottleneck for operations involving many small files.
 - High inode usage can slow down metadata access.
- **Single OST Access:**
 - Large files written to a single OST can saturate bandwidth.
 - Use striping to distribute load across multiple OSTs.
- **Client-Side I/O Patterns:**
 - Serial I/O from many processes can overwhelm shared resources.
 - Optimize with parallel I/O libraries or collective operations.
- **Network Bandwidth:**
 - Shared network links between clients and OSS can become congested.
 - Plan data transfers during off-peak hours or use dedicated paths.
- **File System Contention:**
 - Multiple jobs accessing the same files or directories can cause lock contention.
 - Use job-specific directories and avoid shared file writes.

VAST over NFS

■ Ritom

- \$SCRATCH on selected clusters, usually mounted as `/ritom/cscs/scratch/$USER`
- Quality of Service (QoS) prevents individual users from overloading the system
- Deduplication gives more usable space than Lustre
- Encryption and multi-tenancy for secure computing
- For high-performance applications, specific MPI-IO tuning help

Storage Best Practices

Striping on Lustre: Parallelism for Large Files

■ What is Striping?

- Distributes file data across multiple Object Storage Targets (OSTs).
- Enables parallel read/write operations
 - This is on the Lustre backend, independent of your application using parallel I/O or not!

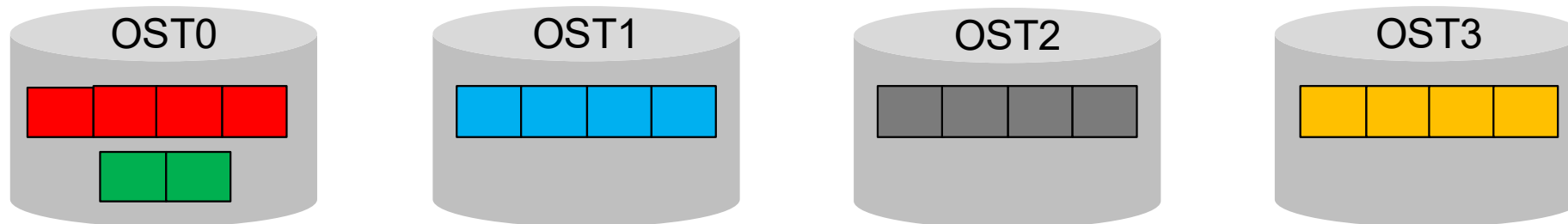
■ Why It Matters

- Crucial for large datasets and high-throughput workflows.
- Reduces bottlenecks in I/O performance.

■ A file is basically a sequence of bytes:



■ On the Lustre FS, the data would be distributed across the various OSTs:



Striping on Lustre: Commands and Examples

Basic command to manage file/folder striping on lustre:

```
# Check current striping
lfs getstripe <directory>

# Set striping for a directory (applies to new files)
lfs setstripe --stripe-count 32 --stripe-size 4M <big_files_dir>
```

Examples

- For improved performance using UENVs
 - `lfs setstripe --stripe-count 20 --stripe-size 1M <uenv.squashfs>`
- For large files
 - `lfs setstripe --stripe-count -1 --stripe-size 4M <big_files_dir>`

Best Practices

- Use `lfs setstripe` for manual control.
- Match stripe count to number of OSS nodes for optimal performance.
- Avoid excessive striping for small files.

Please find more details and examples on the CSCS Documentation:
<https://docs.cscs.ch/storage/#lustre-tuning>

Tuning MPI-IO on Ritom

- Ritom uses VAST over NFS.
 - MPI-IO libraries may apply NFS-specific locking that can slow down collective I/O.
- Settings for MPICH-derived MPI (including Cray MPICH) with HDF5, NetCDF, ADIOS, or checkpoint/restart:

```
export ROMIO_FSTYPE_FORCE='ufs:/ritom/'

export MPIIO_COMMON='romio_cb_read=enable:romio_cb_write=enable:romio_cb_alltoall=enable'
export MPIIO_COMMON="${MPIIO_COMMON}:romio_ds_read=enable:romio_ds_write=enable"
export MPIIO_COMMON="${MPIIO_COMMON}:cb_config_list=#*.*#:cb_nodes=${SLURM_NTASKS:-1}"

export MPICH_MPIIO_HINTS="*:${MPIIO_COMMON}"
```

- What this does:
 - Treats `/ritom/...` as a local Unix filesystem, bypassing NFS lock overhead
 - Enables collective buffering and data sieving for fewer, larger I/O operations
 - Configures aggregator ranks for shared-file writes
- Best for many ranks writing shared files.
 - File-per-rank I/O may see little benefit.
 - Test with your own workload!

You can find more details and examples on the CSCS Documentation:
<https://docs.cscs.ch/guides/storage/#vast-tuning-on-ritom>

Squashfs: Compacting Read-Only Input Data

■ What is squashfs?

- A compressed, read-only file system.
- Packs many small files into a single archive.

■ Why Use It?

- Reduces inode usage and speeds up metadata operations. (critical on Lustre FS)
- Ideal for static input datasets (e.g., simulation configs, reference data).

■ Best Practices

- Create squashfs archives with mksquashfs.
 - Improved version of mksquashfs provided in all UENVs (better compression algorithm)
- Mount can be done using
 - Squashfs-mount
 - Squashfs-use
 - Slurm option
 - `--uenv=<my_uenv>,<my_data.squashfs>:<my_data_mountpoint>`

You can find more details and examples on the CSCS Documentation:
<https://docs.cscs.ch/guides/storage/#many-small-files-vs-hpc-file-systems>

Extra option to address the IOPS bottleneck

The GH200 nodes provide a tmpfs (/dev/shm) in memory

- **Local tmpfs provides ultra-fast, memory-backed storage** for temporary files.
- Ideal for metadata-heavy operations or intermediate data that would otherwise stress Lustre.
 - Staging
 - Pre-processing
 - Compiling an application
 - Caching (c.f. <https://docs.cscs.ch/software/sciapps/cp2k/#dependencies>)
- Reduces I/O contention and latency by offloading from shared file systems.

Limited Resource!

Data Transfer Realities – Understanding the Challenge

Transfer Speed: Theory vs. Reality

- **Theoretical bandwidth $\neq$ actual performance**
- Real-world transfer speed depends on:
 - File size and count
 - Source and destination file systems
 - Network congestion
 - Transfer method (e.g., rsync, scp, tar)
- **Data movement is part of your workflow — plan for it!**
 - Timelines needed for the migration of data sets can easily reach days or weeks

Best Practices for Internal and External Transfers

Internal Transfers (within CSCS systems)

- **Avoid** heavy data movement on login nodes!

- Use the xfer Slurm queue for moving data

```
#SBATCH --partition=xfer
#SBATCH --ntasks=1
#SBATCH --time=02:00:00
srun rsync -av $1 $2
```

- Supports rsync, cp, mv, rm, rclone
- Optimized for performance and scheduling
- Automation and job chaining with compute jobs
 - **Does not consume project resources!**

External Transfers (to/from CSCS)

- Use **Globus** as the primary tool for secure, efficient transfers
- Handles large datasets and supports restart/resume
- May be slower due to external network limits and institutional firewalls

rclone beats rsync at scale

rclone copies in parallel and can be tuned (examples below)

Many files

```
rclone copy --transfers=16 --checkers=32
```

Large files

```
rclone copy --multi-thread-streams=4 --multi-
thread-cutoff=256M --transfers=4
```

For more details and example scripts, please check out the CSCS documentation:

<https://docs.cscs.ch/storage/transfer/>

Key takeaways

Key takeaways

- **Storage is a system**
Architecture and behaviour affect your workflows directly
- **Use the right storage tier for the right task**
Hot, warm, and cold storage serve different roles, plan your workflow accordingly
- **Workflow-level planning is critical**
Optimize data transfers, parallel access, and storage needs
- **Parallel file systems offer performance, but requires awareness**
Lustre striping and VAST over NFS MPI-IO settings for demanding workloads
- **Monitor storage usage**
`quota` command available on the login nodes of production systems

Outlook

A heads-up that the storage landscape at CSCS continues to evolve:

- **datacache** available on the ML platform
 - Available since last maintenance
 - Designed to sit between \$SCRATCH and \$STORE
 - Fast storage providing space to projects needing to share data sets (avoid duplication)
 - **No** cleanup, **no** backup
 - Request by PI via CSCS support portal providing use case as well as capacity and inodes needs
 - Decision based on CSCS review
- CSCS is preparing a new storage system: **Grimsel**
 - Planned to become available to users sometime in 2027
 - Mainly intended as the future home of \$STORE
 - A small reminder why to use \$STORE rather than hardcoded paths!
 - Capacity and performance details are not yet available

Where to read more

■ Storage

- Overview: <https://docs.cscs.ch/storage>
- File systems and quotas: <https://docs.cscs.ch/storage/filesystems>
- Data transfer: <https://docs.cscs.ch/storage/transfer>
- Storage guide: <https://docs.cscs.ch/guides/storage>
- Object storage: <https://docs.cscs.ch/storage/object>

■ Platform specifics

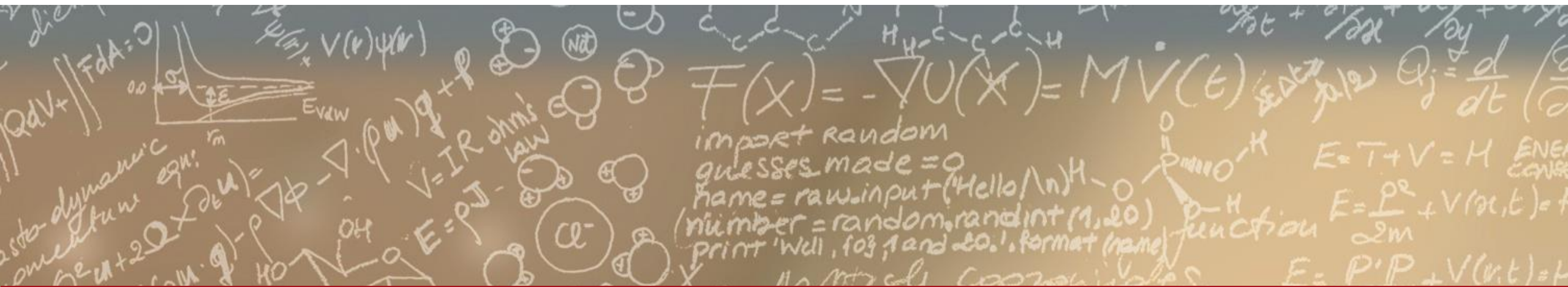
- CWP Platform storage: <https://docs.cscs.ch/platforms/cwp>
- HPC Platform storage: <https://docs.cscs.ch/platforms/hpcp>
- ML Platform storage: <https://docs.cscs.ch/platforms/mlp>



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



Thank you for your attention.